

TECH TEAM KICKOFF

PRAVESH

Campus Event & Ticketing Platform

Team lead: **Swapanth** Squad: 4 engineers Build window: 1 week Tooling: OpenCode

Events. Tickets. Entry. Settlement. — one auditable system.

The manual flow breaks at scale

TODAY

- Google Form collects student details
- Static PhonePe QR shown for payment — no link between form and payment
- Student uploads a payment screenshot + types the transaction ID manually
- Organizer opens the sheet, opens the bank statement, and tallies every single row by hand
- No ticket, no QR, no real check-in — just a filled row

AT 1000 PARTICIPANTS

- Screenshots can be faked or edited — nothing is actually verified
- Organizer time to reconcile grows linearly with every registration
- No dedup — the same row can let two people in
- No audit trail if money or a seat is disputed
- Entry is slow because there's no scannable credential

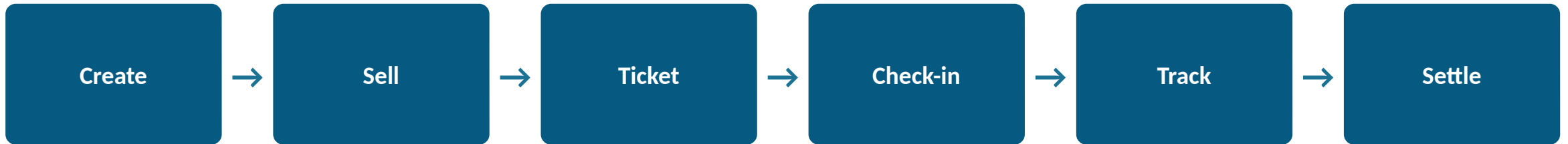
Pravesh, in one sentence

A centralized campus event infrastructure layer that lets clubs publish events and manage tickets, while giving students a fast registration experience — and giving college administration one source of truth for payments, settlements, and entry records.

Not a registration form with a QR code — a campus-wide operating layer.

One lifecycle for every event

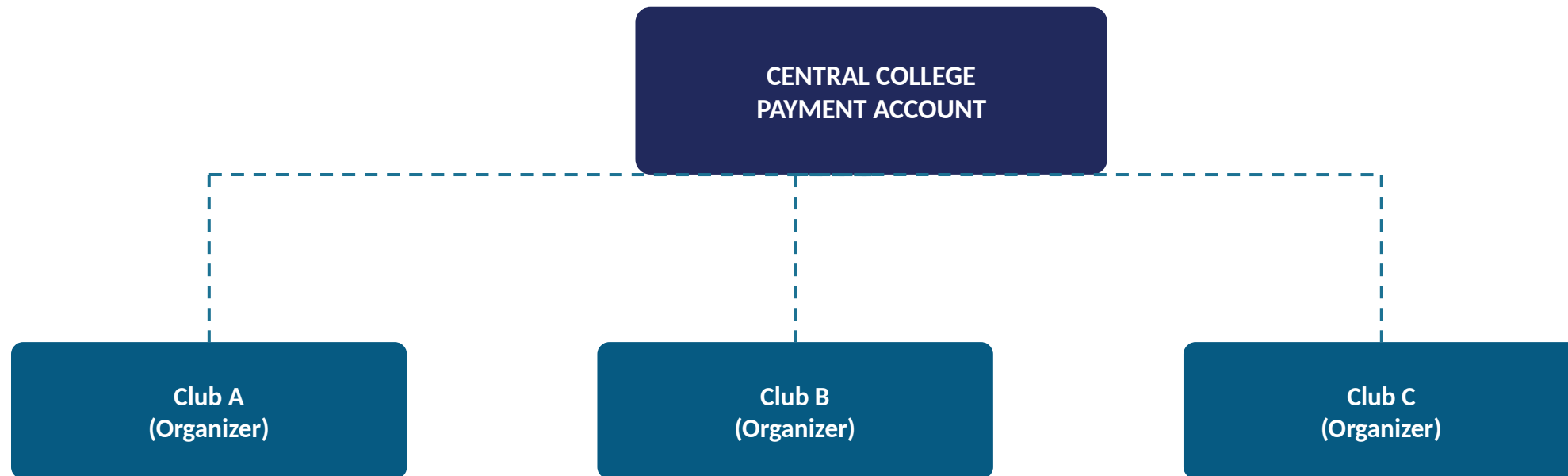
Create once. Sell centrally. Issue a secure ticket. Check in fast. Track every rupee. Settle internally.



- Every stage is owned by the platform, not by an individual club's improvised process — so every event, big or small, behaves the same way

The financial boundary, not the event page

The college is the single financial authority. Clubs run events — money runs through one central, auditable account.



Event ops flow up, verified money flows down — every transaction tagged to exactly one event + organizer.

Rule of thumb: operational ownership follows the event. Financial authority remains with the college.

Roles & permissions

1

STUDENT

- Discover public events
- Register & pay
- Receive ticket + QR
- View own registrations only

2

ORGANIZER

- Create & manage events
- Define ticket types & pricing
- View own attendee & revenue data
- Request payouts

3

ADMIN

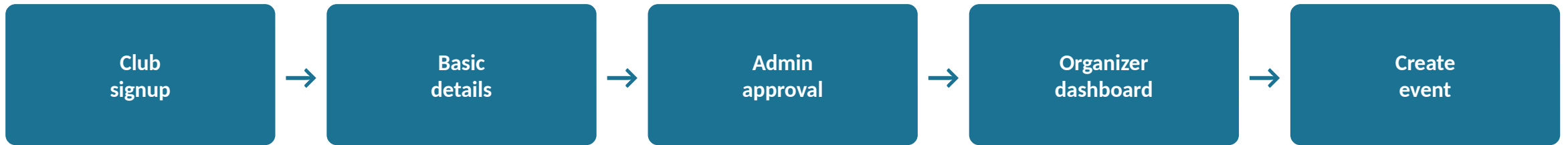
- Approve organizers
- Monitor all payments
- Approve / reject payouts
- Campus-wide financial oversight

What the student experiences



- Ticket + registration are only created after payment is verified — never before
- Student never manually shares a screenshot; the platform confirms payment server-side

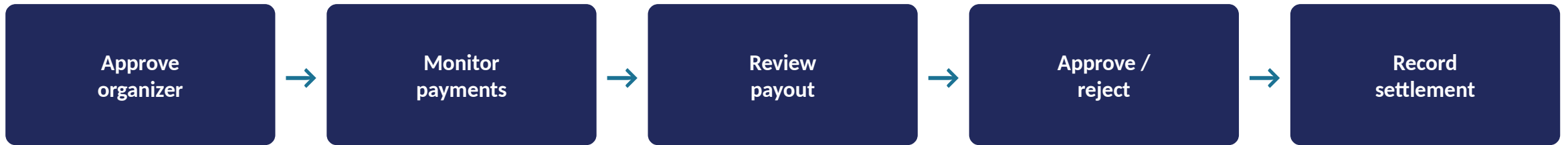
Organizer onboarding & event creation



No payment gateway setup per club — the central account means an organizer moves from approval to a live event fast.

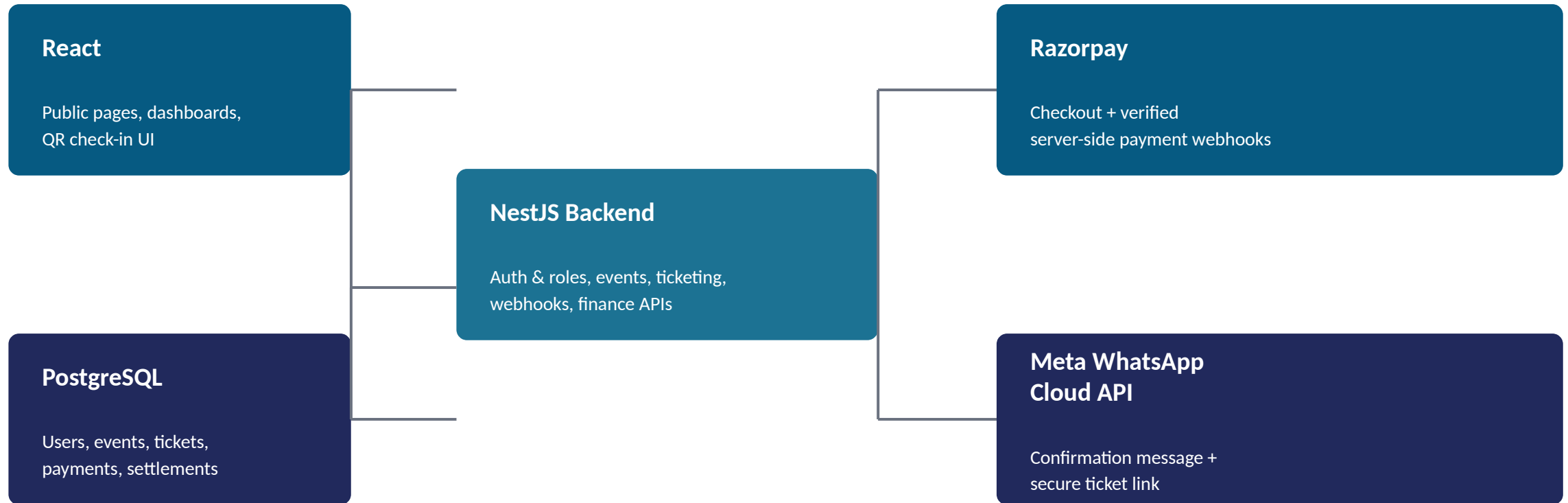
- Event setup captures: title/description, banner, date/venue, capacity, custom registration fields, ticket types (Early Bird / Regular / VIP / Team)
- Draft → preview → publish, so the organizer verifies before it goes public

What the admin controls



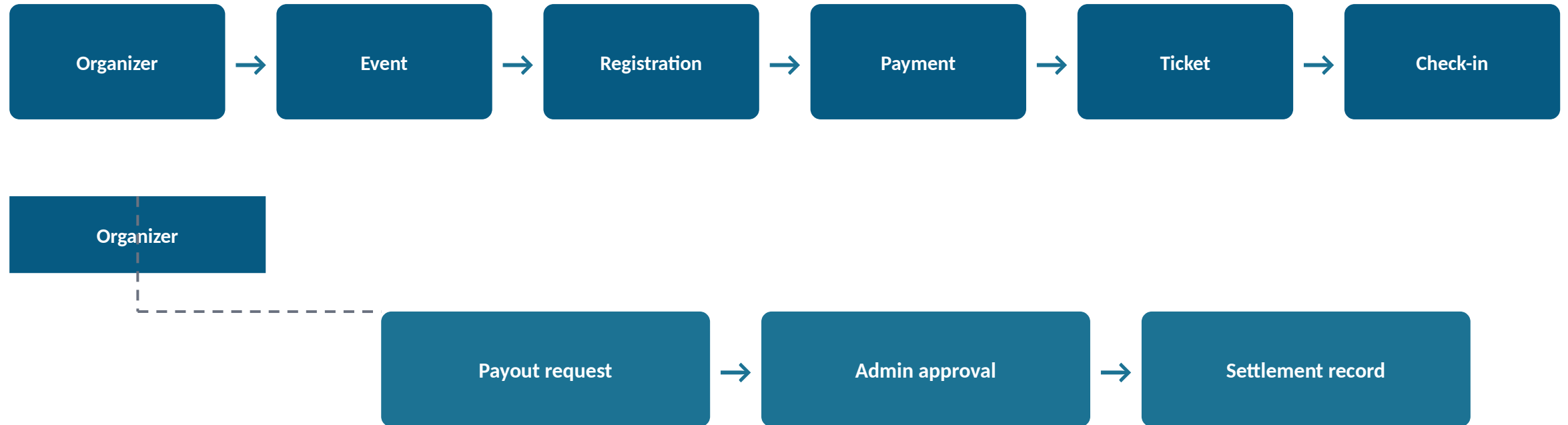
- Reviews revenue breakdown, refunds/adjustments, previous settlements, requested amount, and remaining balance before approving any payout
- Every approval, rejection, and settlement is a recorded, auditable event — nobody edits a balance directly

Architecture — high level



Implementation rule: payment confirmation and ticket creation stay server-side. API secrets never reach the frontend.

Data model — core entities



- Every entity is immutable-append: a balance is never stored as an editable number — it's derived from transaction history
- Payment record always carries: event, organizer, ticket, payment reference, amount, status

Payment & webhook flow



SERVER-SIDE RULES

- Webhooks are authoritative — the frontend is never the source of truth for payment status

CRITICAL EDGE CASE

If Razorpay confirms payment but our app fails before ticket creation, we must safely reconcile the payment and issue the missing ticket — without a duplicate charge or a duplicate ticket. This is our #1 idempotency requirement for week 1.

Ticketing & QR validation

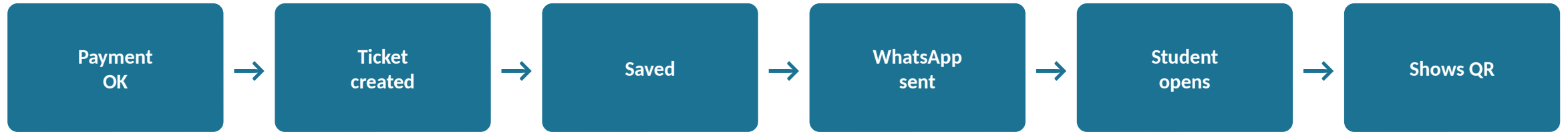
TICKET RECORD

- Ticket number — human-readable unique reference
- QR token — secure identifier for entry validation
- Event ID — binds ticket to one event
- Registration ID — links to the student
- Payment status — determines entry validity
- Check-in status — prevents reuse

CHECK-IN VALIDATION

- Ticket exists for this event
- Payment status is successful
- Not already checked in (no duplicates)
- Not cancelled or refunded
- QR uses a non-guessable / signed token — never exposes attendee data

WhatsApp ticket delivery



IMPLEMENTATION RULES

- Send only after payment is verified AND ticket creation succeeds
- MVP format: secure ticket link + QR (PDF later)
- Access token stays on backend — never in the frontend
- Delivery failure is recorded; ticket stays available in the student dashboard

MESSAGE CONTENT

```
REGISTRATION CONFIRMED
Event: Tech Fest 2026
Ticket ID: PRV-102938
Your digital ticket: pravesh.in/ticket/PRV-102938
Show the QR at the entrance.
```

MVP scope — build this first

MVP — WEEK 1

- Auth + roles
- Event creation & publish
- Public event pages
- Razorpay payment + webhook
- Digital tickets (QR)
- WhatsApp delivery
- QR check-in
- Organizer analytics
- Payout requests
- Admin finance & audit trail

LATER EXPANSION

- Coupons & waitlists
- Certificates
- Team registrations
- Sponsorships
- Automated refunds
- Multiple payment providers
- Offline check-in scanning

Who builds what

Lead — you

Architecture, webhook & ledger design, code review, integration

Teammate 1

Auth & roles, event creation, public event pages

Teammate 2

Razorpay integration, webhook verification, finance & payouts

Teammate 3

Ticketing, QR generation & signing, check-in scanning UI

Teammate 4

WhatsApp Cloud API delivery, organizer dashboard, admin console

One-week build plan (with OpenCode)

Day 1	Schema + auth	DB schema, roles, event CRUD skeleton
Day 2	Event lifecycle	Create/publish, ticket types, public event page
Day 3	Payment core	Razorpay checkout + webhook verification + idempotency
Day 4	Ticket + QR	Ticket issuance, signed QR tokens
Day 5	WhatsApp + check-in	Meta API delivery, QR scan validation
Day 6	Finance + admin	Payout requests, approvals, settlement, audit trail
Day 7	Integration + demo	End-to-end test with 1000-participant load, polish, demo

Use OpenCode for scaffolding, boilerplate CRUD, and test generation — spend human review time on the webhook/idempotency and finance logic.

Risks & edge cases

Webhook retries

Razorpay may resend the same webhook — must be idempotent, no double ticket/charge

Duplicate check-in

Same QR scanned twice must be rejected on the second attempt

Refund handling

A refunded ticket must invalidate check-in eligibility immediately

Cross-club data leakage

An organizer must never see another club's attendees or revenue

Definition of done

- ✓ A real student can register and pay for a real event end-to-end
- ✓ A verified webhook — not the frontend — creates the payment record and ticket
- ✓ The student receives the ticket + QR over WhatsApp automatically
- ✓ A gate scan validates event, payment, and check-in status, and rejects a repeat scan
- ✓ An organizer can request a payout and an admin can approve/reject it with a visible audit trail
- ✓ The flow holds up at a simulated 1000-participant load without manual reconciliation

Questions? Let's build.

Open discussion — architecture, module split, timeline concerns.

PRAVESH